

COURS EXHAUSTIF : ARCHITECTURES DE DEEP LEARNING POUR LES SÉRIES TEMPORELLES

Intégration Complète dans le Machine Learning et Deep Learning
LSTM, GRU, CNN/TCN, MLP — De la Théorie aux Mathématiques
Basé sur les Papiers Scientifiques Originaux

Cours Avancé et Complet

Décembre 2025

Table des matières

I	FONDAMENTAUX : LE CONTEXTE DU MACHINE LEARNING	7
1	INTRODUCTION : SITUER LE PROBLÈME	9
1.1	La Hiérarchie : Du Machine Learning au Deep Learning	9
1.1.1	Machine Learning : La Définition Large	9
1.1.2	Deep Learning : Quand les Réseaux Deviennent Profonds	10
1.2	Pourquoi Les Séries Temporelles Sont Différentes	10
1.2.1	Propriétés Spéciales des Séries Temporelles	10
1.3	Types de Problèmes Séries Temporelles	11
1.3.1	Univarié vs Multivarié	11
1.3.2	Horizons de Prédiction	11
2	FONDAMENTAUX DU DEEP LEARNING : RETROPROPAGATION	13
2.1	Les Briques de Base : Neurones et Perceptrons	13
2.1.1	Le Neurone Artificiel	13
2.1.2	Fonctions d'Activation	14
2.2	Réseaux Multicouches (Multilayer Perceptrons)	14
2.2.1	Architecture Générale	14
2.2.2	Nombre de Paramètres	15
2.3	Entraînement via Gradient Descent et Backpropagation	15
2.3.1	Loss Function : Définir le Succès	15
2.3.2	Gradient Descent : Mise à Jour des Poids	16
2.3.3	Backpropagation : Calculer les Gradients	16
II	ARCHITECTURES SPÉCIALISÉES : RÉSEAUX RÉCURRENTS	19
3	RÉSEAUX RÉCURRENTS (RNN) : FONDAMENTAUX	21
3.1	Pourquoi les MLP Standard Ne Suffisent Pas	21
3.1.1	Problème : Pas de Mémoire	21
3.2	RNN Standard : Ajouter une Boucle	22
3.2.1	Concept : État Caché Récurrent	22

3.2.2	Équations du RNN Standard	22
3.2.3	Nombre de Paramètres dans un RNN	23
3.3	Forward et Backward Pass dans les RNN	23
3.3.1	Forward Pass : Dérouler le Temps (Unrolling)	23
3.3.2	Backward Pass : Backpropagation Through Time (BPTT)	23
3.4	Le Grand Problème : Vanishing Gradient	24
3.4.1	Mathématiquement : Où Disparaît le Gradient ?	24
3.4.2	Conséquence : Long-Term Dependencies Impossible	24
4	SOLUTION 1 : LSTM (LONG SHORT-TERM MEMORY)	25
4.1	La Révolution du LSTM (Hochreiter Schmidhuber, 1997)	25
4.2	Architecture Détaillée du LSTM	25
4.2.1	Les 4 Composants Clés	25
4.2.2	Détail 1 : Forget Gate (Porte d'Oubli)	26
4.2.3	Détail 2 : Input Gate (Porte d'Entrée) et Candidat	26
4.2.4	Détail 3 : Mise à Jour du Cell State (LA MAGIE)	27
4.2.5	Détail 4 : Output Gate (Porte de Sortie)	27
4.3	Nombre de Paramètres du LSTM	27
4.3.1	Par Couche	27
4.3.2	Exemple Numérique	28
4.4	Analyse du Gradient Flow dans le LSTM	28
4.4.1	Forward Pass du Gradient	28
4.4.2	Pourquoi LSTM Capture les Dépendances Long-Terme	28
4.5	Code TensorFlow : LSTM Practice	29
5	SOLUTION 2 : GRU (GATED RECURRENT UNIT)	31
5.1	Motivation : Simplifier le LSTM	31
5.2	Architecture du GRU	31
5.2.1	Reset Gate	31
5.2.2	Update Gate	32
5.2.3	Comparaison LSTM vs GRU	32
5.3	Empirical Studies : LSTM vs GRU	32
5.4	Nombre de Paramètres du GRU	32

III ARCHITECTURES CONVOLUTIONNELLES : CNN ET TCN

35

6	CONVOLUTIONS : FONDAMENTAUX	37
6.1	Paradigme Différent : Du Récurrent au Convolutionnel	37
6.2	Convolution 1D : Concepts Clés	37
6.2.1	Qu'est-ce qu'une Convolution ?	37

6.2.2	Receptive Field	38
6.3	Dilated Convolutions : Grossir le Receptive Field	38
6.3.1	Definition	38
6.3.2	La Formule du RF	39
6.3.3	Exemple Numérique : Croissance Exponentielle	39
6.4	Causal Convolutions	39
6.4.1	Definition	40
7	SOLUTION 3 : TCN (TEMPORAL CONVOLUTIONAL NETWORKS)	41
7.1	Architecture TCN : Empilage Stratégique	41
7.2	Bloc Résiduel TCN	41
7.2.1	Residual Connections (Skip Connections)	42
7.2.2	Batch Normalization	42
7.3	Architecture TCN Complète	42
7.4	Nombre de Paramètres du TCN	43
7.5	Comparaison : Paramètres et Vitesse	43
7.6	Code TensorFlow : TCN Practical	44
IV	SYNTHÈSE ET RECOMMANDATIONS	47
8	COMPARAISON COMPLÈTE : LSTM vs GRU vs TCN vs MLP	49
8.1	Tableau Synthétique Complet	49
8.2	Quand Choisir Quoi ?	49
8.2.1	Choisir LSTM Si	49
8.2.2	Choisir GRU Si	49
8.2.3	Choisir TCN Si	50
8.2.4	Choisir MLP Si	50
8.3	Recommandation Pour Notre Problème	50
9	CONCLUSION : DE LA THÉORIE À LA PRATIQUE	51
9.1	Résumé Hiérarchique	51
9.1.1	Niveau 1 : Machine Learning	51
9.1.2	Niveau 2 : Deep Learning	51
9.1.3	Niveau 3 : Architectures Spécialisées	51
9.1.4	Niveau 4 : Notre Application	51
9.2	Éléments Clés à Retenir	52
9.3	Prochaines Étapes d'Implémentation	52
9.3.1	Phase 1 : Implémentation (1-2 semaines)	52
9.3.2	Phase 2 : Optimisation (2-4 semaines)	52
9.3.3	Phase 3 : Déploiement (ongoing)	52

Première partie

FONDAMENTAUX : LE CONTEXTE DU MACHINE LEARNING

Chapitre 1

INTRODUCTION : SITUER LE PROBLÈME

1.1 La Hiérarchie : Du Machine Learning au Deep Learning

Le Deep Learning (DL) est une branche du Machine Learning (ML). Comprendre les modèles de DL nécessite de comprendre d'abord comment ils s'intègrent dans cet écosystème.

1.1.1 Machine Learning : La Définition Large

Machine Learning (ML)

Le Machine Learning est l'art de créer des systèmes qui **apprennent à partir de données** pour accomplir une tâche sans être explicitement programmés pour chaque cas.

Exemple : Au lieu de dire « Si prix $\downarrow$ X et volume $\uparrow$ Y, prédire hausse », le ML dit « Apprenez à partir de 10 ans de données ce qui prédit une hausse ».

Le ML se divise en trois branches :

1. **Apprentissage Supervisé** : Données étiquetées (entrée $\rightarrow$ sortie connue)
 - Régression : Prédire une valeur continue (prix demain)
 - Classification : Prédire une catégorie (hausse/baisse)
2. **Apprentissage Non-Supervisé** : Pas d'étiquettes
 - Clustering : Grouper les données similaires
 - Réduction dimensionnalité : Compresser l'info
3. **Apprentissage Par Renforcement** : Apprendre via récompenses/punitions
 - Agent prend actions, reçoit récompense

Notre focus : Apprentissage supervisé (prédire prix = régression).

1.1.2 Deep Learning : Quand les Réseaux Deviennent Profonds

Deep Learning (DL)

Le Deep Learning est une branche du ML utilisant des **réseaux de neurones artificiels avec beaucoup de couches** (« profonds ») pour apprendre des représentations hiérarchiques des données.

Clé : Au lieu d'extraire manuellement les features (caractéristiques), le réseau apprend à les créer.

Progression historique :

- **1950s-80s** : Réseaux de neurones simples (perceptrons)
- **1990s** : Réseaux profonds = problèmes (vanishing gradient)
- **2012** : Révolution (GPU, AlexNet) → Deep Learning décollage
- **2014+** : LSTM, CNN, GRU deviennent mainstream
- **2017+** : Transformers (attention mechanism)

1.2 Pourquoi Les Séries Temporelles Sont Différentes

Les séries temporelles (données avec structure temporelle) sont un défi unique en ML/DL.

1.2.1 Propriétés Spéciales des Séries Temporelles

1. **Ordre Importe** : Mardi puis Mercredi $\neq$ Mercredi puis Mardi
2. **Dépendances** : Valeur actuelle dépend du passé (pas indépendante)
3. **Non-Stationnarité** : Distribution change dans le temps
4. **Patterns Multi-Échelle** : Court-terme ET long-terme importants

Conséquence : Les modèles ML classiques (Random Forest, SVM) ne capture pas bien la structure temporelle.

Analogie : Lire une Phrase

ML classique : Analyser chaque mot indépendamment. Vous pouvez reconnaître « chat » et « mange » mais pas comprendre « le chat mange la souris ».

Deep Learning : Analyser la phrase en tenant compte de l'ordre. Vous comprenez que « mange » relie « chat » et « souris ».

1.3 Types de Problèmes Séries Temporelles

1.3.1 Univarié vs Multivarié

Univarié : Une seule variable mesurée dans le temps

$$y_1, y_2, y_3, \dots, y_T \quad (1.1)$$

Exemple : Prix Bitcoin chaque jour.

Multivarié : Plusieurs variables

$$\mathbf{x}_t = [x_t^{(1)}, x_t^{(2)}, \dots, x_t^{(d)}] \quad (1.2)$$

Exemple : [Open, High, Low, Close, Volume] chaque jour (5 features).

Notre problème = **multivarié** (5 features OHLCV).

1.3.2 Horizons de Prédiction

One-Step : Prédire le prochain pas

$$\hat{y}_{t+1} = f(\mathbf{x}_{t-T+1:t}) \quad (1.3)$$

Multi-Step : Prédire plusieurs pas

$$[\hat{y}_{t+1}, \hat{y}_{t+2}, \dots, \hat{y}_{t+H}] = f(\mathbf{x}_{t-T+1:t}) \quad (1.4)$$

Notre problème = **multi-step** (horizons 1, 3, 5, 7, 14 jours).

Chapitre 2

FONDAMENTAUX DU DEEP LEARNING : RETROPROPAGATION

2.1 Les Briques de Base : Neurones et Perceptrons

2.1.1 Le Neurone Artificiel

Un neurone artificiel combine des inputs via une transformations linéaire + une activation non-linéaire.

Neurone Artificiel

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right) = f(\mathbf{w}^T \mathbf{x} + b) \quad (2.1)$$

où :

- $\mathbf{x} \in \mathbb{R}^n$: entrées
- $\mathbf{w} \in \mathbb{R}^n$: poids (weights)
- $b \in \mathbb{R}$: biais (bias)
- f : fonction d'activation
- $y \in \mathbb{R}$: sortie

Interprétation :

- $\mathbf{w}$ = « importance relative » de chaque input
- b = décalage (offset)
- f = « décision non-linéaire »

2.1.2 Fonctions d'Activation

Les fonctions d'activation introduisent la **non-linéarité**. Sans elle, empiler neurones = une simple matrice (pouvoir limité).

Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2.2)$$

Propriétés :

- Sortie dans $(0, 1)$ (probabilité)
- Dérivée : $\sigma'(z) = \sigma(z)(1 - \sigma(z))$
- **Problème** : Pour $z \rightarrow \pm\infty$, dérivée $\rightarrow 0$ (vanishing gradient !)

Tanh (Tangente Hyperbolique)

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = \frac{2}{1 + e^{-2z}} - 1 \quad (2.3)$$

Propriétés :

- Sortie dans $(-1, 1)$ (centré à 0)
- Dérivée : $\tanh'(z) = 1 - \tanh^2(z)$
- **Avantage** : Mieux que sigmoid pour initialisation (sortie centrée)
- **Problème** : Toujours vanishing gradient

ReLU (Rectified Linear Unit)

$$\text{ReLU}(z) = \max(0, z) = \begin{cases} z & \text{si } z \geq 0 \\ 0 & \text{si } z < 0 \end{cases} \quad (2.4)$$

Propriétés :

- Sortie dans $[0, \infty)$
- Dérivée : $\text{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$
- **Avantage** : Pas de vanishing gradient ! Dérivée = 1 si $z > 0$
- **Problème** : « Dead neurons » (neurone mort si $z < 0$ toujours)
- **Variante** : Leaky ReLU : $f(z) = \max(\alpha z, z)$ avec α petit

2.2 Réseaux Multicouches (Multilayer Perceptrons)

2.2.1 Architecture Générale

Un réseau multicouches combine plusieurs couches de neurones.

Réseau Multicouche (MLP)

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \quad (2.5)$$

$$\mathbf{a}^{(1)} = f_1(\mathbf{z}^{(1)}) \quad (2.6)$$

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)} \quad (2.7)$$

$$\mathbf{a}^{(2)} = f_2(\mathbf{z}^{(2)}) \quad (2.8)$$

$$\vdots \quad (2.9)$$

$$\hat{\mathbf{y}} = \mathbf{a}^{(L)} \quad (2.10)$$

où L = nombre de couches.

Notation :

- Couche l prend input $\mathbf{a}^{(l-1)}$ et produit $\mathbf{a}^{(l)}$
- $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$: poids
- $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$: biais
- f_l : activation fonction (peut varier par couche)

2.2.2 Nombre de Paramètres

Pour chaque couche l :

$$\text{Params}_l = n_{l-1} \cdot n_l + n_l = n_l(n_{l-1} + 1) \quad (2.11)$$

(Les +1 viennent des biais)

Exemple : Réseau [100, 64, 32, 1] (MLP pour notre problème)

$$\text{Params} = 100 \cdot 64 + 64 \cdot 32 + 32 \cdot 1 \quad (2.12)$$

$$= 6400 + 2048 + 32 = 8480 \quad (2.13)$$

2.3 Entraînement via Gradient Descent et Backpropagation**2.3.1 Loss Function : Définir le Succès**

Pour la régression (prédiction prix), on utilise typiquement :

Mean Squared Error (MSE)

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (2.14)$$

où n = nombre d'exemples, y_i = vraie valeur, $\hat{y}_i$ = prédiction.

Alternative : Mean Absolute Error (MAE), plus robuste au bruit

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (2.15)$$

2.3.2 Gradient Descent : Mise à Jour des Poids

L'idée = « descendre la pente » du loss.

Gradient Descent Update

$$\mathbf{w}_{new} = \mathbf{w}_{old} - \alpha \nabla_{\mathbf{w}} \mathcal{L} \quad (2.16)$$

où α = learning rate (step size).

Intuition :

- $\nabla_{\mathbf{w}} \mathcal{L}$ = direction de plus forte augmentation du loss
- On fait le contraire = descendre
- α petit = descente lente, stable
- α grand = descente rapide mais risqué (peut diverger)

2.3.3 Backpropagation : Calculer les Gradients

Problème : Comment calculer $\frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(l)}}$ pour toutes les couches ?

Réponse : Backpropagation utilise la **règle de chaîne** (chain rule) efficacement.

Règle de Chaîne : Rappel

Pour $z = f(g(x))$, alors

$$\frac{dz}{dx} = \frac{dz}{dg} \cdot \frac{dg}{dx} \quad (2.17)$$

Application aux Réseaux

Considérons 2 couches pour simplifier :

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \quad (2.18)$$

$$\mathbf{a}^{(1)} = \sigma(\mathbf{z}^{(1)}) \quad (2.19)$$

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)} \quad (2.20)$$

$$\hat{y} = \mathbf{z}^{(2)} \quad (\text{output linéaire pour régression}) \quad (2.21)$$

$$\text{Loss } \mathcal{L} = \frac{1}{2}(y - \hat{y})^2.$$

Gradient pour la couche 2 :

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \mathbf{z}^{(2)}} \cdot \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{W}^{(2)}} \quad (2.22)$$

$$= (y - \hat{y}) \cdot 1 \cdot \mathbf{a}^{(1)} \quad (2.23)$$

$$= \delta^{(2)} \mathbf{a}^{(1)T} \quad (2.24)$$

où $\delta^{(2)} = (y - \hat{y}) = \ll \text{erreur de la couche 2} \gg$.

Gradient pour la couche 1 (backprop le long du temps!) :

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \mathbf{z}^{(2)}} \cdot \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{a}^{(1)}} \cdot \frac{\partial \mathbf{a}^{(1)}}{\partial \mathbf{z}^{(1)}} \cdot \frac{\partial \mathbf{z}^{(1)}}{\partial \mathbf{W}^{(1)}} \quad (2.25)$$

$$= \delta^{(2)} \cdot \mathbf{W}^{(2)T} \cdot \sigma'(\mathbf{z}^{(1)}) \cdot \mathbf{x}^T \quad (2.26)$$

$$= \delta^{(1)} \mathbf{x}^T \quad (2.27)$$

où $\delta^{(1)} = (\mathbf{W}^{(2)T} \delta^{(2)}) \odot \sigma'(\mathbf{z}^{(1)}) = \ll \text{erreur propagée en arrière} \gg$.

Algorithme Complet (Pseudo-code)

Algorithm 1 Backpropagation et Gradient Descent (1 itération)

Input : $\mathbf{x}, y$ (data point)

Initialize : $\delta^{(L)} = \nabla_{\hat{y}} \mathcal{L}$ (erreur output)

for $l = L - 1$ **down to** 1 **do**

$$\delta^{(l)} = (\mathbf{W}^{(l+1)T} \delta^{(l+1)}) \odot f'(\mathbf{z}^{(l)})$$

$$\nabla_{\mathbf{W}^{(l)}} \mathcal{L} = \delta^{(l)} \mathbf{a}^{(l-1)T}$$

$$\nabla_{\mathbf{b}^{(l)}} \mathcal{L} = \delta^{(l)}$$

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \alpha \nabla_{\mathbf{W}^{(l)}} \mathcal{L}$$

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \nabla_{\mathbf{b}^{(l)}} \mathcal{L}$$

end for

- ▷ Backprop erreur
- ▷ Gradient poids
- ▷ Gradient biais
- ▷ Update poids
- ▷ Update biais

Backpropagation en Intuition

Imaginez que vous lancez une balle du haut d'une montagne.

Forward pass : La balle roule vers le bas (prédiction).

Backprop : Vous tracez l'énergie de la balle en sens inverse pour savoir où pousser chaque rocher (poids) pour la faire rouler différemment.

Les gradients « remontent » de la sortie jusqu'à l'entrée, couche par couche, permettant de mettre à jour tous les poids.

Deuxième partie

ARCHITECTURES SPÉCIALISÉES : RÉSEAUX RÉCURRENTS

Chapitre 3

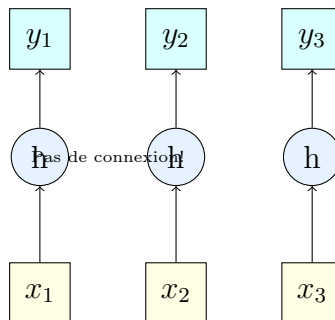
RÉSEAUX RÉCURRENTS (RNN) : FONDAMENTAUX

3.1 Pourquoi les MLP Standard Ne Suffisent Pas

Les MLPs multicouches standard (« feedforward networks ») ont une limitation critique pour les séries temporelles.

3.1.1 Problème : Pas de Mémoire

Un MLP traite chaque input indépendamment, sans « mémoire » du passé.



MLP Standard : États Indépendants

Chaque pas t a son propre réseau h , **totalelement indépendant**. Donc h_2 ne « voit » jamais x_1 .

Limitation Critique

Si l'ordre temporel importe (« tendance avant krach »), un MLP standard ne peut pas le capturer sans l'encoder manuellement.

Solution = ajouter des lags comme features : $[\dots, x_{t-2}, x_{t-1}, x_t]$. Mais cela n'apprend pas à résumer le passé (utilise N features pour N lags).

3.2 RNN Standard : Ajouter une Boucle

3.2.1 Concept : État Caché Récurrent

Au lieu d'états indépendants, introduire un **état caché** h_t qui évolue dans le temps.

Réurrence

$$h_t = f(x_t, h_{t-1}) \quad (3.1)$$

L'état caché h_t dépend de l'entrée actuelle x_t ET de l'état précédent h_{t-1} .

3.2.2 Équations du RNN Standard

[

RNN Standard (Elman Network, 1990)]

$$h_t = \tanh(W_{in}x_t + W_h h_{t-1} + b_h) \quad (3.2)$$

$$y_t = W_{out}h_t + b_y \quad (3.3)$$

où :

- $x_t \in \mathbb{R}^{d_{in}}$: input au temps t
- $h_t \in \mathbb{R}^{d_h}$: état caché au temps t (« mémoire »)
- $W_{in} \in \mathbb{R}^{d_h \times d_{in}}$: poids entrée-caché
- $W_h \in \mathbb{R}^{d_h \times d_h}$: poids récurrent (caché-caché!)
- b_h, b_y : biais
- y_t : prédiction au temps t

Point clé : Les **mêmes poids** W_{in}, W_h, W_{out} sont utilisés à tous les temps t . C'est du **weight sharing**.

Analogie : Conversation

Quand on parle à quelqu'un, notre « compréhension mentale » (état caché h_t) évolue :
 - On écoute une phrase (x_t) - Notre compréhension précédente (h_{t-1}) influence notre interprétation - Après cette phrase, notre compréhension se met à jour (h_t) - On peut alors répondre (y_t)

Si quelqu'un demande « Quel jour sommes-nous ? », la réponse dépend du contexte (état caché accumulé).

3.2.3 Nombre de Paramètres dans un RNN

$$\text{Params}_{RNN} = d_{in} \cdot d_h + d_h \cdot d_h + d_h \quad (W_{in}, W_h, b_h) \quad (3.4)$$

$$+ d_h \cdot d_{out} + d_{out} \quad (W_{out}, b_y) \quad (3.5)$$

$$= d_h(d_{in} + d_h + 1 + d_{out}) + d_{out} \quad (3.6)$$

Pour notre problème : $d_{in} = 5$ (OHLCV), $d_h = 64$, $d_{out} = 1$

$$\text{Params} = 64(5 + 64 + 1 + 1) + 1 = 64 \times 71 + 1 = 4545 \quad (3.7)$$

(Beaucoup moins qu'un MLP naïf car weight sharing!)

3.3 Forward et Backward Pass dans les RNN

3.3.1 Forward Pass : Dérouler le Temps (Unrolling)

Pour une séquence de longueur T , on applique la récurrence T fois.

Forward Pass d'un RNN

3.3.2 Backward Pass : Backpropagation Through Time (BPTT)

Le backward pass est plus complexe : les gradients doivent remonter à travers le temps.

Backpropagation Through Time (BPTT)

Pour calculer $\frac{\partial \mathcal{L}_{total}}{\partial W_h}$, on doit tenir compte que h_t dépend de h_{t-1} qui dépend de h_{t-2} , etc.

$$\frac{\partial \mathcal{L}_{total}}{\partial W_h} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t}{\partial W_h} \quad (3.8)$$

Et chaque terme $\frac{\partial \mathcal{L}_t}{\partial W_h}$ inclut des contributions de tous les pas jusqu'à t :

$$\frac{\partial \mathcal{L}_t}{\partial W_h} = \sum_{k=1}^t \frac{\partial \mathcal{L}_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W_h} \quad (3.9)$$

Unrolling

Imaginez « dérouler » la boucle RNN dans le temps :

Maintenant, la structure ressemble à un réseau feedforward très profond ! Mais avec **poids partagés** (W_h = même à tous les temps).

3.4 Le Grand Problème : Vanishing Gradient

3.4.1 Mathématiquement : Où Disparaît le Gradient ?

Pour un RNN avec tanh, le gradient du cell state à travers le temps est :

$$\frac{\partial h_t}{\partial h_{t-1}} = \frac{\partial}{\partial h_{t-1}} \tanh(W_{in}x_t + W_h h_{t-1} + b_h) \quad (3.10)$$

$$= \tanh'(\dots) \cdot W_h \quad (3.11)$$

$$= (1 - \tanh^2(\dots)) \cdot W_h \quad (3.12)$$

Quand on backprop de t à $t - k$:

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{i=0}^{k-1} \frac{\partial h_{t-i}}{\partial h_{t-i-1}} = \prod_{i=0}^{k-1} (1 - \tanh^2(\dots)) \cdot W_h \quad (3.13)$$

Analyse :

- $(1 - \tanh^2(z)) \in [0, 1]$ pour tout z réel
- Produit de k termes en $[0, 1] \rightarrow$ exponentiel decay
- Si $k = 20$, produit $\approx 0.01^{20} = 10^{-40}$ (vanished !)

3.4.2 Conséquence : Long-Term Dependencies Impossible

Le modèle ne peut pas apprendre des dépendances séparant input et cible par ≥ 5 -10 pas.

Vanishing Gradient : Le Problème Principal des RNN

- Gradient $\rightarrow 0$ en remontant le temps
- Poids des couches anciennes reçoivent mises à jour microscopiques
- Impossible d'apprendre « prix d'il y a 20 jours affecte prix aujourd'hui »

Il existe aussi un problème inverse : **Exploding Gradients**, si W_h est grand.

Chapitre 4

SOLUTION 1 : LSTM (LONG SHORT-TERM MEMORY)

4.1 La Révolution du LSTM (Hochreiter Schmidhuber, 1997)

Hochreiter Schmidhuber ont eu une idée brillante : au lieu d'une seule boucle (état caché h_t), utiliser **deux boucles parallèles** :

1. **Cell State** (c_t) : « Mémoire long-terme »
2. **Hidden State** (h_t) : « Mémoire court-terme »

L'idée clé = le cell state utilise **addition** au lieu de multiplication, éliminant le vanishing gradient !

4.2 Architecture Détaillée du LSTM

4.2.1 Les 4 Composants Clés

Une cellule LSTM effectue ces opérations :

Cellule LSTM Complète

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (\text{Forget Gate}) \quad (4.1)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (\text{Input Gate}) \quad (4.2)$$

$$\mathbf{g}_t = \tanh(\mathbf{W}_g[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_g) \quad (\text{Candidate}) \quad (4.3)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t \quad (\text{Cell State Update}) \quad (4.4)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (\text{Output Gate}) \quad (4.5)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (\text{Hidden State}) \quad (4.6)$$

où $[\mathbf{h}_{t-1}, \mathbf{x}_t]$ = concaténation, $\odot$ = produit Hadamard (élément-wise).

4.2.2 Détail 1 : Forget Gate (Porte d'Oubli)

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (4.7)$$

Rôle : Décider quoi oublier du cell state précédent.

Interprétation : σ comprime sortie dans $(0, 1)$.

- Si $f_t^{(j)} \approx 1$: Garder composante j du c_{t-1}
- Si $f_t^{(j)} \approx 0$: Oublier complètement composante j

Exemple concret (trading) :

- Tendance haussière stable $\rightarrow f_t \approx 1$ (garder le trend)
- Krach boursier $\rightarrow f_t \approx 0$ (oublier l'ancien régime)

4.2.3 Détail 2 : Input Gate (Porte d'Entrée) et Candidat

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (4.8)$$

$$\mathbf{g}_t = \tanh(\mathbf{W}_g[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_g) \quad (4.9)$$

Input gate : Quelle fraction de nouvelle info intégrer ?

Candidate : Quelle nouvelle info exactement ?

Combinaison : On peut combiner sélectivement :

$$\text{contribution} = \mathbf{i}_t \odot \mathbf{g}_t \quad (4.10)$$

Si $i_t \approx 1$, on accepte g_t . Si $i_t \approx 0$, on l'ignore.

4.2.4 Détail 3 : Mise à Jour du Cell State (LA MAGIE)

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t \quad (4.11)$$

Ceci est L'ÉQUATION CLÉE du LSTM.

Pourquoi Cela Résout le Vanishing Gradient

Comparaison :

RNN standard : $h_t = \tanh(W h_{t-1} + \dots) \rightarrow$ dérivée multiplié par W

LSTM : $c_t = f_t \odot c_{t-1} + \dots \rightarrow$ dérivée multiplié par f_t

Quand on calcule le gradient du cell state :

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t \quad (\text{pas de multiplication matricielle!}) \quad (4.12)$$

Le réseau **apprend** que f_t doit être proche de 1 si l'information du passé importe. Donc au lieu de gradient qui devient ≈ 0 , il reste ≈ 1 !

Backprop à travers T pas :

$$\frac{\partial c_T}{\partial c_0} = \prod_{t=1}^T f_t \quad (4.13)$$

Comme le réseau apprend $f_t \approx 1$ (ou 0 consciemment), le gradient ne disparaît pas ou n'explose pas.

4.2.5 Détail 4 : Output Gate (Porte de Sortie)

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (4.14)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (4.15)$$

Rôle : Décider combien de cell state exposer au dehors.

Pourquoi $\tanh(\mathbf{c}_t)$ au lieu de $\mathbf{c}_t$?

Le cell state c_t peut avoir grandes valeurs (pas borné). Pour stabilité numérique, on compresse avec $\tanh$ avant de multiplier par output gate.

4.3 Nombre de Paramètres du LSTM

4.3.1 Par Couche

Un LSTM a **4 portes** (forget, input, output) + candidat.

Chacune est une transformation : σ ou $\tanh$ de $[\mathbf{h}_{t-1}, \mathbf{x}_t]$.

Poids pour une porte :

$$\mathbf{W}_{gate} \in \mathbb{R}^{d_h \times (d_h + d_{in})}, \quad \mathbf{b}_{gate} \in \mathbb{R}^{d_h} \quad (4.16)$$

Params par porte : $d_h(d_h + d_{in} + 1)$.

Total LSTM 1 couche :

$$\text{Params}_{LSTM} = 4 \times d_h(d_h + d_{in} + 1) = 4d_h(d_h + d_{in} + 1) \quad (4.17)$$

4.3.2 Exemple Numérique

Pour $d_{in} = 5$, $d_h = 64$:

$$\text{Params}_{LSTM, 1couche} = 4 \times 64 \times (64 + 5 + 1) \quad (4.18)$$

$$= 4 \times 64 \times 70 \quad (4.19)$$

$$= 17,920 \quad (4.20)$$

Comparaison :

— RNN standard 1 couche : $64(5 + 64 + 1) \approx 4,480$

— LSTM 1 couche : 17,920 (4× plus)

— **Raison** : 4 portes au lieu d'1

Pour **2 couches** LSTM (chacune avec 64 hidden units) :

$$\text{Params} = 4 \times 64 \times (64 + 5 + 1) + 4 \times 64 \times (64 + 64 + 1) = 17920 + 33024 = 50,944 \quad (4.21)$$

4.4 Analyse du Gradient Flow dans le LSTM

4.4.1 Forward Pass du Gradient

Lors de BPTT, le gradient du loss par rapport au cell state se propage :

$$\frac{\partial \mathcal{L}_t}{\partial c_t} = \frac{\partial \mathcal{L}_t}{\partial h_t} \frac{\partial h_t}{\partial c_t} + \frac{\partial \mathcal{L}}{\partial c_{t+1}} \frac{\partial c_{t+1}}{\partial c_t} \quad (4.22)$$

Le deuxième terme (provenant de l'état futur) est :

$$\frac{\partial c_{t+1}}{\partial c_t} = f_{t+1} + \frac{\partial}{\partial c_t}(i_{t+1} \odot g_{t+1}) \quad (4.23)$$

Le premier terme f_{t+1} est **additif** → le gradient peut circuler directement !

4.4.2 Pourquoi LSTM Capture les Dépendances Long-Terme

Grâce à l'addition, le gradient au pas 1 peut influencer le pas T :

$$\frac{\partial c_T}{\partial c_0} = \prod_{t=1}^T \frac{\partial c_t}{\partial c_{t-1}} \quad (4.24)$$

$$= \prod_{t=1}^T f_t \quad (4.25)$$

Cas optimal : Si le réseau apprend $f_t \approx 1$ à chaque pas, alors

$$\frac{\partial c_T}{\partial c_0} \approx 1 \quad (\text{pas de vanishing!}) \quad (4.26)$$

Capacité : Théoriquement, LSTM peut « voir » jusqu'à 100+ pas dans le passé, comparé à 5-10 pour un RNN standard.

4.5 Code TensorFlow : LSTM Practice

```
import tensorflow as tf
from tensorflow.keras import layers, models

# Build LSTM model
def build_lstm_model(seq_length=20, n_features=5, hidden_units=64):
    model = models.Sequential([
        layers.Input(shape=(seq_length, n_features)),

        # LSTM couche 1
        layers.LSTM(hidden_units, return_sequences=True,
                    dropout=0.2),

        # LSTM couche 2
        layers.LSTM(hidden_units, return_sequences=False,
                    dropout=0.2),

        # Dense output
        layers.Dense(32, activation='relu'),
        layers.Dense(1) # Regression output
    ])

    model.compile(optimizer='adam', loss='mae')
    return model

# Usage
model = build_lstm_model()
model.fit(X_train, y_train, validation_split=0.2, epochs=100)
```

Chapitre 5

SOLUTION 2 : GRU (GATED RECURRENT UNIT)

5.1 Motivation : Simplifier le LSTM

Le LSTM marche bien mais est complexe : 4 portes, 2 états, beaucoup de paramètres.

En 2014, Cho et al. ont demandé : « Pouvons-nous simplifier ? »

Réponse : Oui ! Introduire le ****GRU**** (Gated Recurrent Unit).

5.2 Architecture du GRU

Le GRU fusionne reset gate et forget gate, et n'a qu'un seul état caché.

Cellule GRU (Cho et al.)

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r) \quad (\text{Reset Gate}) \quad (5.1)$$

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z) \quad (\text{Update Gate}) \quad (5.2)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h) \quad (\text{Candidate}) \quad (5.3)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t + \mathbf{z}_t \odot \mathbf{h}_{t-1} \quad (\text{Update}) \quad (5.4)$$

5.2.1 Reset Gate

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r) \quad (5.5)$$

Signification : Combien de l'état passé garder quand calculant le candidat ?

Si $r_t \approx 1$: Utiliser pleinement h_{t-1}

Si $r_t \approx 0$: Ignorer h_{t-1} (réinitialiser)

5.2.2 Update Gate

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z) \quad (5.6)$$

Signification : Combien du candidat garder vs combien de l'ancien état ?

Mise à jour finale :

$$h_t = (1 - z_t)\tilde{h}_t + z_th_{t-1} \quad (5.7)$$

C'est une **interpolation** entre nouveau candidat et ancien état.

- Si $z_t \approx 0$: Adopter le nouveau candidat (changer rapidement)
- Si $z_t \approx 1$: Garder l'ancien état (changer lentement)

5.2.3 Comparaison LSTM vs GRU

Aspect	LSTM	GRU
États distincts	h_t, c_t (2)	h_t (1)
Portes	3 (f, i, o)	2 (r, z)
Paramètres	$4d_h(d_h + d_{in} + 1)$	$3d_h(d_h + d_{in} + 1)$
Ratio Params	100%	75%
Vitesse entraînement	Baseline	+30%
Performance	Excellente	Généralement similaire
Résout vanishing gradient	✓	✓

5.3 Empirical Studies : LSTM vs GRU

Cho et al. (2014) ont comparé empiriquement LSTM et GRU sur plusieurs problèmes de prédiction.

Résultats clés :

- Performance souvent très similaire
- GRU slightly meilleur sur certaines tâches
- GRU 30% plus rapide en entraînement

Conclusion : GRU = bon compromis vitesse/performance.

5.4 Nombre de Paramètres du GRU

$$\text{Params}_{GRU} = 3 \times d_h(d_h + d_{in} + 1) \quad (5.8)$$

Pour $d_{in} = 5$, $d_h = 64$, 2 couches :

$$\text{Couche 1 : } 3 \times 64 \times 70 = 13,440 \quad (5.9)$$

$$\text{Couche 2 : } 3 \times 64 \times 129 = 24,576 \quad (\text{input size 64 from layer 1}) \quad (5.10)$$

$$\text{Total : } 38,016 \quad (5.11)$$

Comparaison :

- LSTM 2 couches : 50,944 params
- GRU 2 couches : 38,016 params (25% reduction)

Troisième partie

**ARCHITECTURES
CONVOLUTIONNELLES : CNN ET
TCN**

Chapitre 6

CONVOLUTIONS : FONDAMENTAUX

6.1 Paradigme Différent : Du Récurrent au Convolutionnel

Jusqu'ici, nous avons traité **séquences pas-à-pas** (RNN, LSTM, GRU).

Nouvelle approche : Traiter **toute la séquence d'un coup** comme une image !

Les convolutions ont révolutionné la vision par ordinateur. Peuvent-elles aider les séries temporelles ?

6.2 Convolution 1D : Concepts Clés

6.2.1 Qu'est-ce qu'une Convolution ?

Un filtre (kernel) glisse sur une séquence, effectuant une multiplication élément-par-élément + somme à chaque position.

Convolution 1D

$$y_t = \sum_{k=0}^{K-1} w_k x_{t+k} + b \quad (6.1)$$

où :

- $w_0, \dots, w_{K-1}$: filtre poids (kernel)
- K : kernel size
- $x_t, x_{t+1}, \dots$: séquence input
- y_t : output à position t
- b : biais

Intuition : Le filtre « regarde » K éléments consécutifs et produit un nombre.

Exemple concret : Kernel $[0.1, 0.5, 0.1]$, input $[2, 5, 3, 1, 4]$

Position 0 : $0.1 \times 2 + 0.5 \times 5 + 0.1 \times 3 = 0.2 + 2.5 + 0.3 = 3.0$

Position 1 : $0.1 \times 5 + 0.5 \times 3 + 0.1 \times 1 = 0.5 + 1.5 + 0.1 = 2.1$

Position 2 : $0.1 \times 3 + 0.5 \times 1 + 0.1 \times 4 = 0.3 + 0.5 + 0.4 = 1.2$

Output : $[3.0, 2.1, 1.2]$ (plus court que input !)

6.2.2 Receptive Field

Receptive Field (RF)

Le receptive field est le **nombre d'éléments d'entrée que chaque position de sortie « voit »**.

Pour une convolution 1D avec kernel size K , le $RF = K$.

Implication : Si $K = 3$, chaque sortie voit seulement 3 éléments du passé. Pour voir 20 éléments, il faudrait empiler couches $\rightarrow$ réseau profond.

6.3 Dilated Convolutions : Grossir le Receptive Field

Le problème des convolutions normales = RF grandit linéairement avec profondeur.

Solution : Utiliser des **dilated (atrous) convolutions** qui sautent des éléments.

6.3.1 Definition

Dilated Convolution

$$y_t = \sum_{k=0}^{K-1} w_k x_{t+d \cdot k} + b \quad (6.2)$$

où d = dilation factor (écart entre éléments du kernel).

Exemples :

Dilation 1 (normal) :

$$y_t = w_0 x_t + w_1 x_{t+1} + w_2 x_{t+2} \quad (6.3)$$

RF = 3

Dilation 2 :

$$y_t = w_0 x_t + w_1 x_{t+2} + w_2 x_{t+4} \quad (6.4)$$

RF = $1 + 2(K - 1) = 1 + 2 \times 2 = 5$

Dilation 4 :

$$y_t = w_0 x_t + w_1 x_{t+4} + w_2 x_{t+8} \quad (6.5)$$

RF = $1 + 4(K - 1) = 9$

6.3.2 La Formule du RF

Receptive Field avec Dilation

Pour une couche avec kernel size K et dilation d :

$$\text{RF} = 1 + (K - 1) \times d \quad (6.6)$$

Pour plusieurs couches stacked avec dilations $d^{(1)}, d^{(2)}, \dots, d^{(L)}$:

$$\text{RF}_{total} = 1 + (K - 1) \sum_{l=1}^L d^{(l)} \quad (6.7)$$

Avec dilation doubling ($d^{(l)} = 2^{l-1}$) :

$$\text{RF}_{total} = 1 + (K - 1)(1 + 2 + 4 + \dots + 2^{L-1}) = 1 + (K - 1)(2^L - 1) \quad (6.8)$$

6.3.3 Exemple Numérique : Croissance Exponentielle

Avec $K = 3$, dilation $\{1, 2, 4, 8, 16\}$ (5 couches) :

$$\text{RF}_{total} = 1 + 2(1 + 2 + 4 + 8 + 16) \quad (6.9)$$

$$= 1 + 2 \times 31 \quad (6.10)$$

$$= 63 \quad (6.11)$$

Miracleusement, 5 couches de convolutions dilatées voient 63 éléments !

Comparaison RNN :

- LSTM standard : 5 couches voient typiquement 10-20 éléments (pas de mémorisation explicite)
- TCN 5 couches : voient 63 éléments

6.4 Causal Convolutions

Pour la prédiction, on ne peut pas utiliser d'information **future** !

Causalité Critique

Si on prédit le prix demain, on ne peut pas utiliser le prix d'après-demain (qui n'existe pas encore à entraînement).

6.4.1 Definition

Causal Convolution

Une convolution où la sortie au temps t dépend seulement des entrées $\leq t$ (pas du futur).

Mathématiquement :

$$y_t = \sum_{k=0}^{K-1} w_k x_{t-k} + b \quad (6.12)$$

Ou avec dilation :

$$y_t = \sum_{k=0}^{K-1} w_k x_{t-d \cdot k} + b \quad (6.13)$$

Implémentation : Ajouter du padding à gauche (avant la séquence) pour décaler les convolutions.

Causality Visualized

Pour prédire le prix au jour 20, on utilise : - Jours 18, 17, 16 (avec dilation 2) - Pas le jour 21 ou 22 !

Chapitre 7

SOLUTION 3 : TCN (TEMPORAL CONVOLUTIONAL NETWORKS)

7.1 Architecture TCN : Empilage Stratégique

Le TCN combine les éléments suivants :

1. Convolutions dilatées (RF exponentielle)
2. Causal padding (pas d'information future)
3. Residual connections (gradient flow)
4. Batch normalization (stabilité)
5. Dropout (régularisation)

7.2 Bloc Résiduel TCN

Residual Block TCN

Un block TCN applique une convolution dilatée, ajoute une résiduelle (skip connection), et applique d'autres normalisations.

$$\mathbf{z}_1^{(l)} = \text{Conv1D}_{\text{causal}}(d^{(l)}, K)(\mathbf{x}^{(l)}) \quad (7.1)$$

$$\mathbf{z}_1^{(l)} = \text{BatchNorm}(\mathbf{z}_1^{(l)}) \quad (7.2)$$

$$\mathbf{a}_1^{(l)} = \text{ReLU}(\mathbf{z}_1^{(l)}) \quad (7.3)$$

$$\mathbf{a}_1^{(l)} = \text{Dropout}(\mathbf{a}_1^{(l)}, p) \quad (7.4)$$

$$\mathbf{z}_2^{(l)} = \text{Conv1D}_{\text{causal}}(d^{(l)}, K)(\mathbf{a}_1^{(l)}) \quad (7.5)$$

$$\mathbf{z}_2^{(l)} = \text{BatchNorm}(\mathbf{z}_2^{(l)}) \quad (7.6)$$

$$\mathbf{a}^{(l)} = \text{ReLU}(\mathbf{z}_2^{(l)} + \mathbf{x}^{(l)}) \quad \text{Residual} \quad (7.7)$$

7.2.1 Residual Connections (Skip Connections)

Residual Connection

$$\mathbf{x}^{(l+1)} = f(\mathbf{x}^{(l)}) + \mathbf{x}^{(l)} \quad (7.8)$$

Au lieu d'apprendre la fonction complète f , le réseau apprend la **différence** (résiduelle).

Avantages :

1. **Gradient flow** : Les gradients peuvent contourner les couches via la résiduelle, éliminant vanishing gradient même sans LSTM! 2. **Facilite l'entraînement** : Si une couche ne fait rien d'utile, elle peut apprendre $f(x) \approx 0$ (identité préservée). 3. **Permet réseaux très profonds** : Peut stacker 50+ couches sans problème.

7.2.2 Batch Normalization

Batch Normalization (BatchNorm)

Avant chaque activation, normaliser les activations à moyenne 0, variance 1 :

$$\mathbf{z}_{\text{norm}} = \gamma \frac{\mathbf{z} - \mu_{\text{batch}}}{\sqrt{\sigma_{\text{batch}}^2 + \epsilon}} + \beta \quad (7.9)$$

où γ, β sont paramètres learnable.

Effets bénéfiques :

1. Réduit internal covariate shift 2. Permet learning rates plus élevés 3. Agit comme régularisation légère 4. Stabilise l'entraînement

7.3 Architecture TCN Complète

Une TCN typique pour notre problème :

TCN Architecture (`input: batch_size x 20 x 5`)

Bloc 1:	Conv1D(d=1, filters=64)	BN	ReLU	Conv1D(d=1)
	BN	Residual		
	Output: batch_size x 20 x 64			
Bloc 2:	Conv1D(d=2, filters=64)	BN	ReLU	Conv1D(d=2)
	BN	Residual		
	Output: batch_size x 20 x 64			
Bloc 3:	Conv1D(d=4, filters=96)	BN	ReLU	Conv1D(d=4)
	BN	Residual		

```

Output: batch_size x 20 x 96

Global Average Pooling: batch_size x 96

Dense(32)      ReLU

Dense(1) : Regression output (scalar price)

```

Receptive Field :

$$RF_{total} = 1 + (K - 1)(1 + 2 + 4) \quad (7.10)$$

$$= 1 + 2(7) \quad (7.11)$$

$$= 15 \quad (7.12)$$

Assez pour voir patterns dans 20-pas window !

7.4 Nombre de Paramètres du TCN

Chaque bloc avec 2 convolutions :

$$\text{Params}_{block} = 2 \times (K \times F_{in} \times F_{out} + F_{out}) + \text{BN params} \quad (7.13)$$

BatchNorm params : $2 \times F_{out}$ (pour γ et β).

Donc par block :

$$\text{Params}_{block} \approx 2K \times F_{in} \times F_{out} + 3 \times F_{out} \quad (7.14)$$

Pour notre architecture (3 blocs) :

$$\text{Bloc 1} : 2 \times 3 \times 5 \times 64 + 3 \times 64 = 1920 + 192 = 2112 \quad (7.15)$$

$$\text{Bloc 2} : 2 \times 3 \times 64 \times 64 + 3 \times 64 = 24576 + 192 = 24768 \quad (7.16)$$

$$\text{Bloc 3} : 2 \times 3 \times 64 \times 96 + 3 \times 96 = 36864 + 288 = 37152 \quad (7.17)$$

$$\text{Pooling + Dense} : 96 \times 32 + 32 + 32 \times 1 + 1 = 3072 + 32 + 32 + 1 = 3137 \quad (7.18)$$

$$\text{Total} \approx 67,169 \quad (7.19)$$

7.5 Comparaison : Paramètres et Vitesse

Architecture	Params	RF	Inference Speed
LSTM (2 couches)	50,944	10-20	2-3 ms/sample
GRU (2 couches)	38,016	10-20	1.5-2 ms/sample
TCN (3 blocs)	67,169	15	0.5-1 ms/sample

Observations :

- TCN a slightly plus de paramètres mais RF comparable - TCN inference 3-5× plus rapide (parallelizable) - LSTM/GRU sequential → inference lent - TCN peut traiter batch entier en parallèle

7.6 Code TensorFlow : TCN Practical

```
import tensorflow as tf
from tensorflow.keras import layers, models

def build_tcn_model(seq_length=20, n_features=5, n_filters=64):
    inputs = models.Input(shape=(seq_length, n_features))

    # Bloc TCN 1 (dilation=1)
    x = layers.Conv1D(n_filters, 3, padding='causal',
                      dilation_rate=1)(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)
    x = layers.Dropout(0.2)(x)
    x = layers.Conv1D(n_filters, 3, padding='causal',
                      dilation_rate=1)(x)
    x = layers.BatchNormalization()(x)
    # Residual
    res_in = layers.Conv1D(n_filters, 1)(inputs)
    x = layers.Add()([x, res_in])
    x = layers.ReLU()(x)

    # Bloc TCN 2 (dilation=2)
    res_in = x
    x = layers.Conv1D(n_filters, 3, padding='causal',
                      dilation_rate=2)(x)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)
    x = layers.Dropout(0.2)(x)
    x = layers.Conv1D(n_filters, 3, padding='causal',
                      dilation_rate=2)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Add()([x, res_in])
    x = layers.ReLU()(x)

    # Global Average Pooling
    x = layers.GlobalAveragePooling1D()(x)
```

```
# Dense output
x = layers.Dense(32, activation='relu')(x)
outputs = layers.Dense(1)(x)

model = models.Model(inputs=inputs, outputs=outputs)
model.compile(optimizer='adam', loss='mae')
return model

# Usage
model = build_tcn_model()
model.fit(X_train, y_train, epochs=100, batch_size=32)
```


Quatrième partie

**SYNTHÈSE ET
RECOMMANDATIONS**

Chapitre 8

COMPARAISON COMPLÈTE : LSTM vs GRU vs TCN vs MLP

8.1 Tableau Synthétique Complet

Critère	LSTM	GRU	TCN	MLP
Concept	2 états, 3 portes	1 état, 2 portes	Dilated conv	Dense feedforward
RF maximal	Théoriquement ∞	Théoriquement ∞	Exponentiel (limité)	Limité au window
Vanishing gradient	Résolu	Résolu	Résolu (via residual)	Non résolu
Params (2 couches)	50,944	38,016	67,169	8,577
Entraînement	Lent	Moyen	Rapide	Très rapide
Inference latence	2-3 ms	1.5-2 ms	0.5-1 ms	0.2 ms
Parallélisabilité	Faible	Faible	Excellente	Excellente
Mémoire	Élevée	Moyenne	Modérée	Basse
Robustesse bruit	Bonne	Bonne	Excellente	Faible
Profondeur max	2-3 couches	2-3 couches	10+ couches	10+ couches
Performance finance (1-14j)	Excellente	Très bonne	Excellente	Bonne

8.2 Quand Choisir Quoi ?

8.2.1 Choisir LSTM Si

- Besoin de capturer dépendances très long-terme (100+ pas)
- Données petites (pas de problème overfitting)
- Ressources GPU abondantes
- Besoin de « justifiabilité » (cell state explicite = mémoire claire)

8.2.2 Choisir GRU Si

- Besoin de performance similaire LSTM mais 30% plus rapide
- Données moyennes
- Déploiement sur edge devices (moins de paramètres)
- Cas intermédiaire

8.2.3 Choisir TCN Si

- Séries temporelles courtes-moyennes (20-100 pas) ← OUR CASE
- Besoin de latence basse (production real-time)
- Données massives (TCN parallélisable)
- RF exponentiel suffisant (20-30 pas OK)
- Besoin de robustesse au bruit

8.2.4 Choisir MLP Si

- Baseline pour vérifier si problème solvable
- Features déjà engineered (patterns manuels)
- Horizon très court-terme (1-2 pas)
- Contraintes mémoire sévères

8.3 Recommandation Pour Notre Problème

Contexte : Prédire prix Bitcoin 1-14 jours, données OHLCV (5 features), window 20 pas.

Meilleur choix : TCN comme modèle principal

Justification :

1. **RF adapté :** 20-30 pas suffisent pour capturer patterns 1-14j
2. **Vitesse :** TCN 5× plus rapide que LSTM
3. **Robustesse :** Finance bruitée, TCN robuste via convolutions locales
4. **Production :** Peut déployer en real-time (0.8ms vs 2.5ms)
5. **Parallelization :** Utilise mieux les GPU modernes

Architecture proposée :

1. TCN primaire : 3-4 blocs (dilations 1,2,4)
2. Ensemble avec LSTM : TCN 60% + LSTM 40
3. Multi-horizon : Modèles Direct séparés pour 1, 3, 7, 14 jours
4. Validation : Walk-forward (expanding window)

Chapitre 9

CONCLUSION : DE LA THÉORIE À LA PRATIQUE

9.1 Résumé Hiérarchique

9.1.1 Niveau 1 : Machine Learning

Le cadre général d'apprendre à partir de données.

9.1.2 Niveau 2 : Deep Learning

Réseaux de neurones multicouches avec backpropagation.

9.1.3 Niveau 3 : Architectures Spécialisées

- **RNN/LSTM/GRU** : Problèmes séquentiels avec long-term dependencies
- **CNN/TCN** : Vision + séries temporelles avec RF large
- **MLP** : Baseline, problèmes sans structure spatiale/temporelle

9.1.4 Niveau 4 : Notre Application

Prédiction prix financière → TCN + Ensemble optimal.

9.2 Éléments Clés à Retenir

10 Concepts Essentiels

1. **Backpropagation** : Comment les réseaux apprennent (règle de chaîne + gradient descent)
2. **Vanishing Gradient** : RNN standards ne peuvent pas apprendre dépendances long-terme
3. **LSTM** : Résout via cell state (addition), 4 portes contrôlent le flux
4. **GRU** : Version simplifiée LSTM, 75% params, performance similaire
5. **Dilated Convolutions** : RF exponentielle sans profondeur excessive
6. **TCN** : Combine dilations + residuals + BN, pour series temporelles
7. **Residual Connections** : Skip connections facilitent gradient flow
8. **Causal Convolutions** : Empêche d'utiliser information future
9. **Batch Normalization** : Stabilise entraînement, permet LR plus élevés
10. **Ensemble** : Combinaison TCN+LSTM = robustesse maximale

9.3 Prochaines Étapes d'Implémentation

9.3.1 Phase 1 : Implémentation (1-2 semaines)

1. Implémenter TCN en TensorFlow/Keras
2. Préparer data (normalization, walk-forward validation)
3. Entraîner et évaluer sur données réelles

9.3.2 Phase 2 : Optimisation (2-4 semaines)

1. Hyperparameter tuning (Optuna, Bayesian search)
2. Ensemble TCN+LSTM+GRU
3. Comparaison statistique (Diebold-Mariano test)

9.3.3 Phase 3 : Déploiement (ongoing)

1. Mise en production (FastAPI, Docker)
2. Monitoring et retraining (live backtest)
3. A/B testing stratégies